

作業ログ

報告者：塩澤 匠生

報告日：2026 年 6 月 1 日

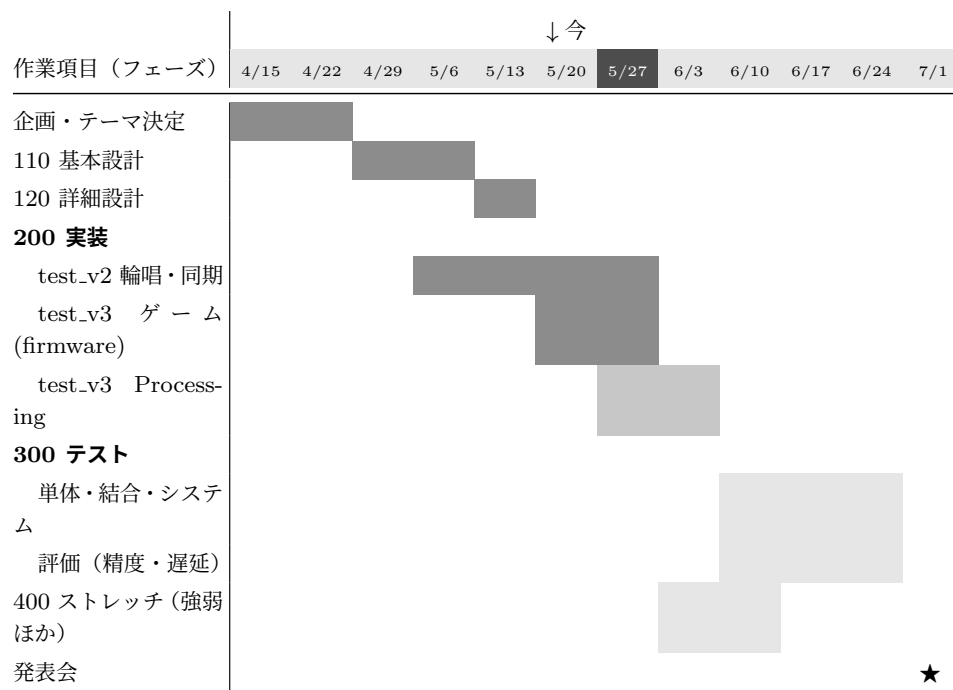
プロジェクト名：タクトーン IMU ジェスチャーで奏でる Arduino オーケストラ

1. 進捗サマリー（要約）

- 全体進捗率：50%（実装フェーズの後半. test_v2 の 3 声輪唱が安定し, その応用として test_v3 「ゲームモード」の実装に着手）
 - マイルストーン達成状況：
 - test_v2 の低遅延化・パケロス分散・WiFi 再 join 堅牢化：達成（PR #21）
 - test_v3 ゲームモードの firmware（共通／node_01／node_02）：達成（PR #22, 実機評価は次週）
 - test_v3 の Processing 画面（メニュー／ゲーム／結果／アナライザ）：進行中
 - 今週の主要成果：
 - test_v2 を低遅延化（発音先読み 50 → 30ms・音声バッファ短縮）し, 連送の時間分散と WiFi 復帰時の再 join で取りこぼしに強くした
 - test_v3 ゲームモードを新規実装：IMU ナビ（左右の振り＝メニュー移動／縦振り＝決定）, テンポ維持精度の採点（0～100 点）, メトロノームガイドの自動フェード
 - 楽器ノードに UI 中継パケット（新 type=4）を追加し, PC 側 Processing がゲーム画面を描けるようにした
 - 重大課題（影響度：中）：
 - [IMU ナビの判定軸・符号・しきい値は実機の振り方に依存するため, 実機書き込みでの調整が必要（設定値で差し替え可能にしてある）]
-

2. 進捗ガントチャート（計画と現在地）

チームのガントチャート（meetings/**ガントチャーター** ver1 ト.xlsx）のフェーズ区分（110 基本設計／120 詳細設計／200 実装／300 テスト／400 ストレッチ）に, 現在の進み具合を重ねたもの. 「今」の縦位置は本報告時点（6 月第 1 週）.



凡例：■ 完了 ■ 進行中 ■ 予定 ★ 発表会（7/1）

※ test_v3 のゲームモードは計画上の 400 ストレッチに相当する追加機能だが、自由演奏（200 実装）が安定したため前倒しで着手した。テスト（300）は次フェーズ。

3. 作業ログ（詳細トラッキング）

作業項目	開始日	予定完了日	状態	進捗	コメント
test_v2 低遅延化・堅牢化	2026-05-27	2026-06-01	完了	100%	発音先読み 50 → 30ms・連送の時間分散・WiFi 再 join. PR #21
test_v3 ゲームモード設計	2026-05-30	2026-06-01	完了	100%	CTRL 予約 4B を mode / navCursor / targetBpm / score にフィールド化
test_v3 firmware 実装	2026-06-01	2026-06-01	完了	100%	IMU ナビ・採点・UI 中継. 全 5 ノード pio run 成功. PR #22
test_v3 Processing 画面	2026-06-02	2026-06-09	進行中	30%	メニュー／ゲーム／結果／アナライザの画面遷移を実装中
実機評価・IMU ナビ調整	2026-06-02	—	未着手	—	振りの軸・符号・しきい値を実機でチューニング

4. 課題管理

課題	発生原因	影 響 度	優 先 度	対策	状態
IMU ナビの判定が 実機依存	振り方（軸・大きさ）が個人 やボード取付向きで変わる ため、机上の値では誤判定し 得る	中	高	しきい値・軸・符号を設定値 (config) に出し、実機書き込 みで調整できるようにした	対応中
UDP マルチキャス トのパケロス	マルチキャストは ACK・再 送が無く、取りこぼしが音抜 けになる	中	中	連送 4 発を 2ms 間隔で時間 分散し、WiFi 復帰時にマル チキャスト購読を貼り直す 堅牢化を実装（先週からの継 続）	改善
5 台構成（金管 4 + ドラム）の輪唱位相	等間隔で 4 声以上重ねると 周期が一巡せず位相が破綻 する	低	低	不等間隔の位相か曲の周期 延長＝編曲（音楽判断）が必 要	保留

※影響度＝プロジェクトへのインパクト，優先度＝対応の緊急性

5. AI 利用状況と検証（再現性重視）

5.1 利用内容

- 利用目的：ゲームモードの設計レビュー（CTRL 予約バイトのフィールド割当，IMU ナビの判定方式，UI 中継パケットの設計）と firmware の実装支援
- 入力（プロンプト要旨）：
 - [同期経路（BEAT / NOTE / CTRL の 20 バイト固定）を壊さずにゲーム用の情報を載せる方法]
 - [左右・縦の振りだけでメニュー操作と決定を行う，誤爆しにくい判定方式]
 - [楽器ノードから PC へ画面用の情報を低頻度で中継する仕組み]
- 出力概要：
 - [既存 CTRL の予約 4 バイトを mode / navCursor / targetBpm / score に割り当て，20 バイトを維持する案]
 - [動的加速度の左右成分でカーソル移動・縦成分で決定とし，一度戻るまで再発火しない不応期を設ける案]
 - [新 type=4 パケットで CTRL 内容を変化時＋最大 5Hz で USB シリアルへ中継する案]

5.2 検証方法・ファクトチェック

- 検証手法：

- [全5ノードを `pio run` でビルドし, RAM / Flash 使用率を確認(node_01 RAM13.8% / Flash21.0% ほか, いずれも従来と同程度)]
 - [同期経路(BEAT / NOTE / CTRL の 20 バイト構造)が不変であることをコードと `static_assert` で確認]
 - 最終判断：
 - 採用（自由演奏は `test_v2` と同一挙動を維持しつつ, ゲームモードを上重ねる構成）。ただし IMU ナビと採点の手応えは実機でしか確認できないため, 軸・しきい値の調整と評価は実機作業に委ねる
-

6. 次回計画

- 目標：
 - [test_v3 の Processing 側 4 画面（メニュー／自由演奏／ゲーム／結果）＋アナライザを実装し, ゲームの一連の流れを通す]
 - [実機書き込みで IMU ナビの軸・しきい値を調整し, 採点の手応えを確認する]
 - 達成基準：
 - [メニュー→ゲーム→結果の遷移が PC 画面で破綻なく動く]
 - [自由演奏モードが `test_v2` と同等の安定性で鳴る]
 - 期限：
 - [6 月中旬までに Processing 画面と実機調整を終え, テストフェーズ (300) へ移行]
-

7. その他

- [ゲームモードは追加機能（ストレッチ）であり, 遅れても自由演奏のみで発表は成立する位置づけ. コア機能の上に重ねている]
 - [実機未テストのファームウェアへの机上修正は同期挙動を逆転させるリスクがあるため, 必ず実機書き込みでの確認を経て取り込む]
-

8. 付録

- 添付資料：
 - ソースコード：firmware/test_v3/（共通ライブラリ＋ node_01～node_04）, pc_app/test_v3/
 - GitHub リポジトリ：<https://github.com/takushio2525/2026-hackathon-team23>
 - 関連 PR：#21（test_v2 低遅延化）, #22（test_v3 ゲームモード）
 - 設計判断記録：ADR-0002（プロトコル選定）, ADR-0003（IMU 採用）, ADR-0006（同期誤差 20ms）
- 添付範囲：

- 差分（test_v2 の低遅延化と test_v3 ゲームモード firmware を実装した直近時点のスナップショット）
-